

Quick Start Guide (ESP32 + Android App)

Contents

1	Quick Start Guide (ESP32 + Android App)	2
1.1	What You Need	2
1.2	Step 1: Flash the ESP32 Firmware	2
1.2.1	Install Arduino IDE + ESP32 boards	2
1.2.2	Open and upload the firmware	3
1.2.3	Flashing recommendations (important)	3
1.3	Step 2: Pair the Phone with the ESP32 (Bluetooth SPP)	4
1.4	Step 3: Connect Using the Android RC App	4
1.5	Step 4: Verify the ESP32 is Receiving Commands (Debug-Focused)	4
1.5.1	Open Serial Monitor	4
1.5.2	Enable firmware debug output (recommended)	5
1.5.3	What you should see (expected debug checkpoints)	5
1.5.4	Fast isolation tests (when something is wrong)	5
1.5.5	Optional: debug without real hardware	6
1.6	Step 5: First Hardware Test (Safe Test Without Motors)	6
1.7	Step 6: Connect Real Hardware	6
1.8	Common Problems (Fast Fixes)	6
1.8.1	App says connected, but nothing moves	6
1.8.2	ESP32 resets when motors start	6
1.8.3	Bluetooth device not visible	7
1.9	Next Steps	7

Chapter 1

Quick Start Guide (ESP32 + Android App)

This chapter is a fast, practical guide to get the system working end-to-end: **flash the ESP32 firmware**, **pair/connect the Android app**, and **verify control + telemetry**.

1.1 What You Need

- ESP32 development board + USB cable (data cable).
- Arduino IDE (1.8.x supported, 2.x recommended).
- The ESP32 firmware project (ESP32_BT_Controller).
- The Android RC Controller application (APK/store/internal deployment).
- Optional (recommended for testing): 1 LED + 330 Ω resistor (to test PWM/switch outputs without motors).

1.2 Step 1: Flash the ESP32 Firmware

1.2.1 Install Arduino IDE + ESP32 boards

- Install Arduino IDE.
- In Arduino IDE, add the ESP32 boards URL:

`https://dl.espressif.com/dl/package\_esp32\_index.json`

- Install **ESP32 by Espressif Systems** in Boards Manager.
- Recommended core: **ESP32 Core Version 2.0.x**.

1.2.2 Open and upload the firmware

- Download Code: Download the project source code from the GitHub repository:

`https://github.com/MICSBOL/ESP32\_BT\_Controller`

You can download the project by clicking the **Code** button and selecting **Download ZIP**, or by cloning the repository using Git.

After downloading the project, extract the files to a folder on your computer.

- Open:

`ESP32_BT_Controller.ino`

- Select board:

`Tools -> Board -> ESP32 Dev Module`

- Select port:

`Tools -> Port -> (your ESP32 port)`

- Click **Upload**.

1.2.3 Flashing recommendations (important)

For the most reliable uploads:

- Flash the ESP32 with **no external hardware connected** (disconnect motors, servos, relays, sensors).
- Avoid anything connected to **boot/strapping pins** during flashing (common offenders: GPIO0, GPIO2, etc.).
- If upload fails, hold **BOOT** when the upload begins (board-dependent).
- If you see random failures, try another USB cable (many cables are charge-only).

1.3 Step 2: Pair the Phone with the ESP32 (Bluetooth SPP)

- Power the ESP32 and ensure it is running the firmware.
- On Android, enable Bluetooth.
- If Android requests permissions (e.g., “Nearby devices” / Location-related permissions), grant them.
- Pair with the ESP32 Bluetooth device (commonly named ESP32-BT).
- If prompted for a PIN, try the default:

1234

1.4 Step 3: Connect Using the Android RC App

- Open the RC Controller app.
- Select the ESP32 device from the available Bluetooth device list.
- Once connected, the UI controls (sticks, knobs, switches, event buttons) become active.

1.5 Step 4: Verify the ESP32 is Receiving Commands (Debug-Focused)

This step is the fastest way to confirm the full chain is working: **Android UI → Bluetooth SPP link → ESP32 packet decode → control logic**.

1.5.1 Open Serial Monitor

- In Arduino IDE, open:

Tools -> Serial Monitor

- Set baud rate:

115200

- Reset the board (or re-plug USB) so you can see boot messages from the beginning.

1.5.2 Enable firmware debug output (recommended)

If your firmware build supports debug flags, enable them in:

```
debug_config.h
```

Then **re-upload** the firmware.

Recommended debug signals to enable If the project offers multiple debug categories, prioritize:

- **Bluetooth / connection state:** confirms SPP connects and stays connected.
- **Receiver / packet decode:** confirms headers are found and packets parse correctly.
- **RC state values:** confirms stick/knob/switch numbers change when you move controls.
- **Event IDs:** confirms button presses are being received as one-shot events.
- **Output writes (HAL):** confirms PWM/switch outputs are being written (useful when hardware “does nothing”).

1.5.3 What you should see (expected debug checkpoints)

Use this as a quick mental checklist:

1. **Boot messages** appear after reset (confirms serial is correct and firmware is running).
2. **Bluetooth ready / advertising** message (wording depends on firmware).
3. **Client connected** when you connect from Android.
4. **RC state updates** when you move sticks/knobs or toggle switches.
5. **Event prints** when you press an event button.

If you see steps 1–3 but not 4–5, the app is connecting but not sending control packets (or the receiver is not decoding them). If you see 4–5 but hardware does not respond, the problem is likely in the output mapping/wiring.

1.5.4 Fast isolation tests (when something is wrong)

- **No serial output at all:** wrong port, wrong baud rate, or firmware not running.
- **Serial output OK but no Bluetooth connect:** phone not paired, permissions missing, wrong device selected, or ESP32 not advertising.
- **Connected but no RC values change:** app not sending, receiver not decoding, or protocol mismatch (old firmware vs new app).
- **RC values change but motors/servos do nothing:** pin mapping mismatch, wrong project mode, missing common ground, or power rail problem.

1.5.5 Optional: debug without real hardware

For early testing, avoid motors/servos and validate outputs with LEDs first:

- Use an LED + $330\ \Omega$ resistor on a switch pin to confirm switch toggles.
- Use an LED on a PWM pin to confirm brightness changes with joystick movement.

1.6 Step 5: First Hardware Test (Safe Test Without Motors)

Before connecting motors/servos, confirm outputs with LEDs:

- Connect an LED + $330\ \Omega$ resistor to a switch output pin (or a PWM pin).
- Toggle a switch in the app: the LED should turn on/off.
- Move a joystick axis mapped to PWM: the LED brightness should change (PWM test).

1.7 Step 6: Connect Real Hardware

Once the control path is verified:

- Connect your motor driver (do **not** drive motors directly from ESP32 pins).
- Connect servos to PWM pins, but power servos from a **separate 5V regulator** sized for peak current.
- Always use **common ground** between ESP32, motor driver, servo supply, and sensors.

1.8 Common Problems (Fast Fixes)

1.8.1 App says connected, but nothing moves

- Ensure you selected the correct ESP32 device (if multiple are nearby).
- Verify firmware upload succeeded and you are not running an old sketch.
- Use Serial Monitor debug output to confirm packets arrive and values change.

1.8.2 ESP32 resets when motors start

- This is usually power sag/noise.
- Use separate power rails/regulators for ESP32 vs motors/servos.
- Add capacitors near the motor driver/servo rail (e.g., 470–1000 μF).

1.8.3 Bluetooth device not visible

- Power-cycle the ESP32.
- Keep the phone within a few meters.
- Check Serial Monitor for Bluetooth startup messages.

1.9 Next Steps

After the quick start works, the typical next steps are:

- Customize pin mapping and project mode (`pins.h`, `user_config.h`).
- Add telemetry sources (ADC / I2C sensors) and verify panels/indicators/plots.
- If you need more digital I/O, switch to MCP23017 expansion mode.